

MultiAgentBench: A Multi-Agent LLM Research and Evaluation Framework

Technical Seminar Literature Review & Systematic Survey

1. Introduction

Large Language Models (LLMs) have emerged as a foundational paradigm in artificial intelligence, transforming natural language processing and general reasoning capabilities [1]. Built upon the Transformer architecture introduced by Vaswani et al. [2], LLMs leverage self-attention mechanisms to process sequential data and capture long-range contextual dependencies across high-dimensional token representations. The dominant pre-training and fine-tuning paradigm—exemplified by BERT [3], GPT-2 [4], GPT-3 [5], and PaLM [6]—demonstrates that self-supervised pre-training on massive text corpora endows models with broad general knowledge, which can subsequently be adapted to specialized tasks through supervised fine-tuning or instruction tuning [7].

Empirical investigations into model scaling have established fundamental scaling laws governing LLM performance [8], [9]. Kaplan et al. [8] demonstrated that cross-entropy loss scales as a power law with respect to parameter count, dataset size, and compute budget. Hoffmann et al. [9] refined these findings with the Chinchilla scaling laws, highlighting the equal importance of scaling tokens alongside parameters. Crucially, as parameter scale exceeds critical thresholds, models exhibit emergent abilities—such as few-shot learning, multi-step arithmetic, symbolic manipulation, and complex instruction following—that are absent in smaller models [10]. These capabilities marked a shift from narrow task-specific modeling to general-purpose language foundation models.

2. Evolution of LLM-Based Agents

The evolution of language modeling has witnessed a paradigm shift from passive next-token prediction to active, goal-directed agentic behavior [11], [12]. Traditional LLMs function primarily as stateless pattern completers, responding to isolated prompts without persistent state, dynamic environmental interactions, or real-time feedback loops. In contrast, LLM-based autonomous agents use the core language model as a central reasoning engine (or "brain"), integrated with external modules for perception, long-term memory, planning, and tool execution [13].

As formalized by Weng [13] and expanded by Xi et al. [12], an LLM agent architecture typically comprises four structural components:

Brain / Reasoning Engine: The underlying LLM responsible for parsing goals, synthesizing contextual information, deliberating over action pathways, and formulating decisions.

Perception Module: Mechanisms that convert multimodal or textual environmental signals, system feedback, and inter-agent messages into structured contextual inputs.

Memory Module: Dual-tier context architectures comprising short-term working memory (in-context conversation buffers) and long-term memory (vector stores, semantic databases, or explicit symbolic

records) [14].

Action / Tool Space: Executable APIs, functions, search engines, or physical actuators enabling the agent to affect state changes in its environment [15].

This architectural synthesis enables agents to operate in dynamic, non-deterministic environments, iteratively pursuing complex goals through multi-step execution loops.

3. Reasoning and Agentic Workflows

A central requirement for autonomous agency is deliberative reasoning. Early zero-shot and few-shot prompting techniques often failed on complex reasoning tasks due to the feed-forward, single-pass generation nature of standard LLM inference. To overcome this limitation, Wei et al. [16] introduced Chain-of-Thought (CoT) prompting, demonstrating that instructing LLMs to generate intermediate step-by-step reasoning paths before producing a final answer significantly enhances performance on mathematical, symbolic, and commonsense reasoning tasks.

Subsequent research expanded CoT into broader agentic workflows. Kojima et al. [17] showed that CoT reasoning can be elicited in a zero-shot manner simply by appending the directive "Let's think step by step" to the prompt, revealing latent step-by-step reasoning mechanisms within instruction-tuned LLMs. To address symbolic computation limitations in pure natural language reasoning, Gao et al. [18] introduced Program-Aided Language Models (PAL), and Chen et al. [19] proposed Program-of-Thought (PoT). These frameworks offload symbolic manipulation, arithmetic, and logical state transitions to external programmatic interpreters (e.g., Python execution environments), treating code generation as an intermediate reasoning step.

4. Autonomous LLM Agents

Building upon step-by-step reasoning, autonomous agent frameworks introduced closed-loop execution cycles capable of self-directed task management without human intervention [11]. Open-source implementations such as AutoGPT [20] and BabyAGI [21] demonstrated early operational paradigms for autonomous execution loops. These agents recursively generate sub-tasks, prioritize task queues, execute actions via web browsing or local code execution, evaluate outcomes, and dynamically update their internal task lists until a top-level goal is achieved.

Concurrently, research into agent memory mechanisms advanced the longevity and coherence of autonomous operations. Park et al. [22] introduced Generative Agents, establishing an architecture that combines a retrieval-based memory stream with reflection mechanisms. In this paradigm, agents continuously log raw observations into a temporal memory stream, retrieve relevant experiences based on recency, importance, and relevance, and periodically synthesize higher-level abstractions ("reflections") to guide future behavior. These memory architectures prevent context window degradation and enable agents to maintain long-term behavioral consistency across extended multi-step trajectories [13], [22].

5. Multi-Agent LLM Systems

While single-agent systems demonstrate impressive capabilities on isolated tasks, they encounter severe cognitive and operational bottlenecks when scaled to complex, multi-faceted problems. Single agents suffer from context window saturation, role drift, catastrophic forgetting, and an inability to perform parallel specialization [23], [24]. These limitations catalyzed the transition toward Multi-Agent LLM Systems (MAS), wherein multiple specialized agents collaborate, communicate, and debate within structured topological networks [23]–[27].

Multi-agent paradigms leverage the principle of modular division of labor [25]. By assigning distinct system prompts, functional tools, and specialized agent personas (e.g., Planner, Researcher, Analyst, Critic, Developer), complex goals are decomposed into targeted sub-problems. Frameworks such as AutoGen [23], CAMEL [24], MetaGPT [25], and AgentVerse [26] demonstrate that multi-agent teams achieve superior problem-solving performance, higher factual consistency, and greater robustness compared to monolithic single-agent baselines.

6. Multi-Agent Collaboration Architectures

The interaction topology governing inter-agent communication defines how information flows, how sub-tasks are delegated, and how collective decisions are synthesized [27], [28]. Literature identifies five canonical collaboration architectures:

6.1 Single-Agent Architecture

The single-agent baseline serves as the minimal reference architecture. A single LLM instance processes the task prompt, executes internal reasoning loops (e.g., CoT or ReAct), invokes tools sequentially, and produces the final output [15], [16]. While computationally efficient and free of inter-agent communication overhead, single-agent architectures lack external critique mechanisms and are susceptible to early reasoning errors propagating uncorrected to final outputs.

6.2 Chain Architecture

The Chain topology organizes agents in a linear, sequential pipeline $A_1 \rightarrow A_2 \rightarrow \dots \rightarrow A_n$, where the output of agent A_i becomes the direct input prompt for agent A_{i+1} [24], [25]. This architecture reflects assembly-line processing or Standard Operating Procedures (SOPs). Sequential pipelines excel in step-by-step document refinement, software generation pipelines (e.g., Specification $\rightarrow$ Architecture $\rightarrow$ Code $\rightarrow$ Documentation), and multi-stage translation tasks. However, Chain architectures are vulnerable to error accumulation along the pipeline and lack lateral feedback loops.

6.3 Star / Centralized Architecture

The Star topology features a centralized Manager or Coordinator agent A_c surrounded by specialized worker agents $A_1, A_2, \dots, A_k$ [23], [26]. Communication channels exist primarily between the central manager and individual workers ($A_c \leftrightarrow A_i$), with direct worker-to-worker communication restricted. The manager decomposes the top-level goal, assigns sub-tasks to specialized workers, gathers worker results, and synthesizes final outputs. Star architectures provide centralized oversight and clear task allocation, but the central node can become a latency and context processing bottleneck.

6.4 Tree / Hierarchical Architecture

The Tree topology structures agents in a multi-tier hierarchy, where root and intermediate manager nodes delegate tasks down to subordinate branch nodes and leaf-level execution agents [27]. Information and sub-task solutions flow back up the tree through aggregation nodes. Hierarchical structures are well-suited for large-scale enterprise modeling, complex software engineering projects, and recursive problem decomposition, as they prevent any single node from being overwhelmed by global context.

6.5 Graph-Based Architecture

The Graph topology generalizes inter-agent communication to an arbitrary directed graph $G = (V, E)$, where nodes V represent specialized agents and edges E denote permitted communication channels [27], [28]. Graph architectures support peer-to-peer collaboration, dynamic message routing, iterative multi-agent debate, and cyclic critique loops. While offering maximal operational flexibility and robust consensus formation, unconstrained graph architectures incur high token consumption, elevated API costs, and increased risk of infinite communication loops.

7. Planning, Critique, Reflection and Debate

Advanced deliberative capabilities in LLM agents rely on explicit search algorithms, self-reflection mechanisms, and multi-agent debate protocols [29]–[34].

To extend step-by-step reasoning into non-linear exploration, Yao et al. [29] introduced Tree of Thoughts (ToT), enabling LLMs to explore multiple reasoning paths across a tree structure, evaluate intermediate states, and use search algorithms (e.g., breadth-first or depth-first search) with lookahead and backtracking. Besta et al. [30] generalized this concept into Graph of Thoughts (GoT), representing LLM thoughts as network vertices and operations (transformation, aggregation, refinement) as edges, thereby supporting arbitrary non-linear thought combination.

For post-execution self-correction, Shinn et al. [31] introduced Reflexion, a framework that equips agents with verbal reinforcement learning. Rather than updating neural weights, Reflexion agents generate explicit verbal critiques of their past execution failures, store these critiques in a memory buffer, and condition subsequent execution attempts on past lessons learned. Similarly, Madaan et al. [32] proposed Self-Refine, demonstrating iterative solution improvement through self-generated feedback loops. Gou et al. [33] introduced CRITIC, showing that agents can self-correct more accurately when critique loops are grounded in interaction with external tools (e.g., code execution outputs, search engine responses).

In multi-agent settings, debate protocols improve factual accuracy and reduce hallucination [34], [35]. Du et al. [34] demonstrated that when multiple LLM instances independently formulate solutions and iteratively debate opposing views, the collective consensus yields higher factual precision and logical consistency than any individual model's output. Li et al. [24] implemented communicative debate through cooperative role-playing in CAMEL, while Zhang et al. [35] formalized Exchange-of-Thought (EoT) protocols for multi-agent negotiation.

8. Tool Use and Agent Interaction

Autonomous agents must interact with external software environments to obtain real-time information and perform side-effecting operations [15], [36]–[38].

Yao et al. [15] proposed ReAct (Reasoning and Acting), establishing a unified framework that interleaves reasoning trace generation ("Thought") with action execution ("Action") and environment feedback collection

("Observation"). By embedding action execution directly within the deliberative reasoning loop, ReAct agents dynamically adjust execution plans based on external environmental observations.

Schick et al. [36] introduced Toolformer, proving that language models can self-teach tool usage by inserting special API calls into text sequences in a self-supervised manner. To scale tool usage to real-world software ecosystems, Qin et al. [37] developed ToolLLM and ToolBench, facilitating master-level interaction across thousands of real-world REST APIs via Neural API Retrievers. Furthermore, Patil et al. [38] introduced Gorilla, demonstrating fine-tuned open-source models capable of accurate API function calling with strict schema adherence, mitigating hallucinated tool arguments.

9. Evaluation and Benchmarking of LLM Agents

As LLM agents transitioned from simple prompt-response models to complex multi-step systems, traditional static NLP evaluation benchmarks (e.g., MMLU, GSM8K, HumanEval) proved insufficient [39]–[43]. Evaluating agents requires interactive, multi-turn environments that assess task execution over extended horizons.

Early single-agent benchmark suites introduced structured environment interactions. Liu et al. [39] created AgentBench, evaluating LLMs across eight distinct environments including OS shells, databases, web browsing, and digital games. Jimenez et al. [40] developed SWE-bench, benchmarking agents on real-world GitHub issue resolution within complex Python codebases. Zhou et al. [41] introduced WebArena, assessing end-to-end web navigation across e-commerce, content management, and forum platforms. Mialon et al. [42] proposed GAIA, targeting general-purpose multi-modal assistant capabilities, while Xie et al. [43] introduced OSWorld for open-ended operating system interaction.

Multi-agent evaluation frameworks emerged to address collaborative dynamics. Chen et al. [26] developed AgentVerse, assessing multi-agent problem solving across cooperative coding, negotiation, and game simulations. Recently, Zhu et al. [28] published MultiAgentBench (powered by the MARBLE execution engine), evaluating LLM multi-agent systems across collaborative and competitive scenarios using key performance metrics and coordination graphs.

10. Evaluation Metrics

Comprehensive agent evaluation demands multi-dimensional metric suites spanning task success, system efficiency, and operational reliability [28], [39]–[44]:

- **Task Success & Milestone Score:** Measures binary completion of top-level objectives as well as granular sub-goal achievement (milestone tracking) [28], [40].
- **Reasoning Performance & Accuracy:** Assesses logical correctness, step verification, and absence of hallucinated facts in agent reasoning chains [16], [34].
- **Constraint Satisfaction Rate:** Evaluates whether agent outputs strictly comply with explicit domain, budget, temporal, or operational constraints specified in the prompt [28].
- **Token Consumption & Financial Cost:** Quantifies total prompt tokens, completion tokens, and dollar expenditures accrued across multi-turn agent turns [23], [25].
- **Execution Latency & Turn Efficiency:** Tracks end-to-end task duration, per-turn response latency, and network round-trip overhead [27], [28].
- **Reliability & Failure Metrics:** Quantifies system-level stability by tracking API failures, rate limit breaches, JSON schema errors, infinite loop traps, and request timeouts [23], [44].

11. Reliability and Failure Modes

Operating LLM agents in production reveals significant failure modes that undermine system reliability [23], [44], [45]. Key failure taxonomies documented in literature include:

Context Drift and Role Collapse: Over extended multi-turn interactions, agents gradually lose context fidelity, drift away from initial system prompt personas, or forget foundational directives [22], [44].

Infinite Communication Loops: In cyclic or unconstrained graph topologies, agents often enter non-terminating repetitive exchange patterns, repeatedly echoing minor variations of the same response [23], [27].

Cascading Hallucination Propagation: In sequential chain or hierarchical architectures, an uncorrected hallucination generated by an upstream agent is treated as grounded truth by downstream agents, leading to compound system failure [24], [34].

API and Schema Violations: Agents frequently generate malformed JSON, invalid function arguments, or invalid parameter types, triggering execution exceptions [38], [44].

Rate-Limit and Provider Timeouts: High-frequency multi-agent API calls saturate provider concurrency quotas, incurring rate-limit failures, HTTP 429 status errors, and unhandled request timeouts [23].

12. Comparative Analysis of Existing Research

To summarize the state of the art, the following comparative tables synthesize existing literature across agent paradigms, coordination topologies, and evaluation frameworks.

Table 1: Comparative Analysis of Representative LLM Agent Studies

Research	Year	Approach	Architecture	Main Contribution	Evaluation	Limitation
Wei et al. [16]	2022	Chain-of-Thought Prompting	Single-Agent	Elicits step-by-step reasoning via intermediate prompts.	GSM8K, SVAMP	Lacks environment feedback & tool interaction.
Yao et al. [15]	2023	ReAct Framework	Single-Agent	Synergizes reasoning traces with environment actions.	HotpotQA, ALFWorld	Susceptible to compounding single-agent errors.
Shinn et al. [31]	2023	Reflexion	Single-Agent + Memory	Verbal reinforcement learning via self-reflection buffers.	HumanEval, WebShop	High context usage; single-agent bottleneck.
Yao et al. [29]	2024	Tree of Thoughts (ToT)	Single-Agent Search	Deliberate non-linear search (BFS/DFS) over thought trees.	Game of 24, Creative Writing	Expensive search compute; no multi-agent roles.
Wu et al. [23]	2023	AutoGen	Multi-Agent Conversational	Customizable conversational agent framework.	HumanEval, MATH	Lacks standardized benchmark suite across topologies.

Research	Year	Approach	Architecture	Main Contribution	Evaluation	Limitation
Li et al. [24]	2023	CAMEL	Multi-Agent Role-Playing	Inception prompting for autonomous agent cooperation.	Communicative tasks	Unconstrained dialogue can drift or loop.
Hong et al. [25]	2024	MetaGPT	Multi-Agent SOP-based	Encodes Standard Operating Procedures into agent roles.	Software tasks	Fixed linear pipeline limit; non-flexible graph.
Chen et al. [26]	2023	AgentVerse	Multi-Agent Dynamic	Environment for dynamic group assembly and debate.	Coding, Negotiation	Limited systematic tracking of API/token cost.
Du et al. [34]	2023	Multi-Agent Debate	Multi-Agent Consensus	Multi-agent consensus debate for factual correctness.	MMLU, Translation	High token latency; non-hierarchical topology.
Zhu et al. [28]	2025	MultiAgentBench / MARBLE	Multi-Agent Benchmark	Unified benchmark across collaborative/competitive tasks.	Multi-scenario benchmark	Complex simulation setup; heavy engine dependency.

Table 2: Architectural Comparison of Inter-Agent Topologies

Architecture	Coordination Style	Advantages	Limitations	Representative Research
Single-Agent	Isolated execution	Zero inter-agent overhead; minimal latency per step.	Context saturation; single point of failure; no peer critique.	Wei et al. [16], Yao et al. [15]
Chain	Linear sequential pipeline	Clear step-by-step progression; simple SOP mapping.	Cascading error propagation; no reverse feedback loops.	Hong et al. [25], Li et al. [24]
Star / Centralized	Hub-and-spoke delegation	Centralized oversight; clear role delegation.	Coordinator node bottleneck; high coordinator context load.	Wu et al. [23], Chen et al. [26]
Tree / Hierarchical	Multi-tier abstraction	Scalable sub-goal decomposition; low local context load.	Deep tree latency; complex routing and aggregation logic.	Besta et al. [30]
Graph-Based	Peer-to-peer network	Dynamic interaction; maximum flexibility; strong consensus.	High token cost; high latency; risk of infinite loop traps.	Zhu et al. [28], Du et al. [34]

Table 3: Synthesis of Evaluation Dimensions Across Benchmark Frameworks

Evaluation Framework	Task Success	Reasoning	Token / Cost	Latency	Reliability & Failure Analysis	Multi-Topology Comparison
AgentBench [39]	High	Medium	Low	Low	Low	No (Single-agent focused)
SWE-bench [40]	High	High	Low	Low	Low	No (Single-agent focused)
WebArena [41]	High	Medium	Low	Low	Low	No (Single-agent focused)
AgentVerse [26]	High	Medium	Low	Low	Medium	Partial (Dynamic group focus)
MultiAgentBench [28]	High	High	Medium	Medium	High	Yes (Multi-scenario focus)

13. Research Gap

Despite rapid advancements in multi-agent LLM systems, a critical synthesis of existing literature reveals several significant research gaps:

Lack of Controlled Multi-Topology Comparisons: Most existing multi-agent frameworks (e.g., MetaGPT, CAMEL) are tightly coupled to a single fixed interaction topology (e.g., sequential pipelines or unconstrained conversational groups). There is a paucity of empirical research that systematically evaluates identical benchmark tasks across **Single, Star, Chain, Tree, and Graph topologies** under strictly controlled prompt budgets and base model configurations.

Absence of Standardized Fine-Grained Operational Metrics: While current benchmarks report high-level task completion rates, comparatively less attention has been given to systematically quantifying fine-grained system operational metrics—specifically **token cost trade-offs, step latency distributions, constraint satisfaction rates, and API error frequencies**.

Inadequate Failure Mode Diagnostics: Existing benchmark suites primarily measure success versus failure but fail to systematically categorize and log runtime failure modes, such as **agent communication loops, schema parsing failures, rate-limit stalls, and context drift**.

Dependence on Complex, Non-Reproducible Environments: Benchmark suites often rely on heavy simulation engines (e.g., Minecraft environments, complex web servers) that introduce significant setup complexity, non-deterministic execution noise, and high computational barriers for lightweight, reproducible research.

14. Relevance to MultiAgentBench

To address these identified research gaps, **MultiAgentBench** is introduced as a comprehensive research and evaluation framework designed for systematic comparative analysis of LLM multi-agent architectures.

MultiAgentBench establishes a modular execution architecture that explicitly isolates interaction topology from base model capability. The framework enables controlled benchmarking across five canonical topologies:

- **SINGLE:** Single-agent baseline execution loop.
- **CHAIN:** Sequential pipeline with step-by-step handoff.

- **STAR:** Centralized coordinator delegating to specialized worker nodes.
- **TREE:** Multi-tier hierarchical sub-task decomposition.
- **GRAPH:** Configurable mesh network supporting peer-to-peer critique and consensus.

Furthermore, MultiAgentBench implements a multi-dimensional metric collection engine that systematically evaluates:

Task Performance: Task success rate, milestone completion score, and reasoning accuracy.

Operational Efficiency: Total token usage (prompt and completion), financial cost estimation, and end-to-end latency.

System Reliability & Guardrails: Constraint satisfaction rate, API failure rates, rate-limit errors, invalid JSON schema responses, request timeouts, and automated agent-loop detection.

By replacing opaque game engines with a transparent, reproducible benchmark task suite spanning planning, constraint satisfaction, synthesis, reasoning, decision making, collaboration, and coding, MultiAgentBench provides researchers with an empirical testbed to evaluate the exact cost-performance-reliability trade-offs inherent in multi-agent orchestration.

15. Summary

This literature review has synthesized the architectural and empirical evolution of Large Language Model agents—tracing the trajectory from foundational Transformer models and chain-of-thought reasoning to autonomous tool-using agents and complex multi-agent ecosystems. While multi-agent collaboration topologies (Chain, Star, Tree, Graph) offer substantial performance gains over single-agent baselines, they introduce complex trade-offs involving token consumption, latency overhead, and systemic failure modes. MultiAgentBench directly addresses the critical research gaps in current evaluation literature by providing a standardized, multi-topology, multi-metric evaluation framework to systematically benchmark the next generation of LLM multi-agent systems.

References

- [1] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pp. 4171–4186, 2019.
- [4] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
- [5] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R.

- Lowe, "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 27730–27744, 2022.
- [6] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, P. Schuh, K. Shi, S. Tsvyashchenko, J. Maynez, A. Rao, P. Barnes, Y. Tay, N. Shazeer, V. Prabhakaran, E. Reif, N. Du, B. Hutchinson, R. Pope, J. Bradbury, J. Ni, C. Nohl, B. Lu, M. Mishra, F. Nguyen, A. Chen, M. Shukla, L. Lerer, C. Heng, S. Zelinka, A. Shukla, A. Nieto, T. Tyler, M. Rainone, P. Pramod, A. Zheng, R. Aroca, R. Ganapathy, D. Zhou, and Q. V. Le, "PaLM: Scaling language modeling with Pathways," *Journal of Machine Learning Research (JMLR)*, vol. 24, no. 240, pp. 1–113, 2023.
- [7] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," in *International Conference on Learning Representations (ICLR)*, 2022.
- [8] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.
- [9] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre, "Training compute-optimal large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 30016–30030, 2022.
- [10] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, E. H. Chi, T. Hashimoto, O. Vinyals, P. Liang, J. Dean, and W. Fedus, "Emergent abilities of large language models," *Transactions on Machine Learning Research (TMLR)*, 2022.
- [11] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, L. Tang, X. Zhang, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
- [12] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Q. Zhou, W. Weers, Y. Zhang, Z. Liu, and Q. Zhang, "The rise and potential of large language model based agents: A survey," *arXiv preprint arXiv:2309.07864*, 2023.
- [13] L. Weng, "LLM-powered autonomous agents," *Weng's Blog*, 2023. [Online]. Available: <https://lilianweng.github.io/posts/2023-06-23-agent/>
- [14] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, pp. 1–22, 2023.
- [15] S. Yao, J. Zhao, D. Yu, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.
- [16] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 24824–24837, 2022.
- [17] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 22199–22213, 2022.
- [18] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, "PAL: Program-aided language models," in *International Conference on Machine Learning (ICML)*, pp. 10764–10799, 2023.
- [19] W. Chen, X. Ma, X. Wang, and W. W. Cohen, "Program of Thought prompting: Disentangling computation from reasoning for numerical reasoning tasks," *Transactions on Machine Learning Research (TMLR)*, 2023.

- [20] T. Richards, "AutoGPT: An autonomous GPT-4 experiment," 2023. [Online]. Available: <https://github.com/Significant-Gravitas/AutoGPT>
- [21] K. Nakajima, "BabyAGI: Task-driven autonomous agent," 2023. [Online]. Available: <https://github.com/yoheinakajima/babyagi>
- [22] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Social simulacra: Creating populated online spaces using computer-generated personas," in *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology (UIST)*, pp. 1–13, 2022.
- [23] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, L. Tan, and H. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.
- [24] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for 'mind' exploration of large language model society," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 51991–52008, 2023.
- [25] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations (ICLR)*, 2024.
- [26] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y. Dong, L. Liu, M. Zhang, and J. Li, "AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors in agents," in *International Conference on Learning Representations (ICLR)*, 2024.
- [27] C. Qian, X. Cong, C. Yang, W. Chen, Y. Su, J. Xu, Z. Liu, and M. Sun, "Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 14889–14907, 2024.
- [28] K. Zhu, H. Du, Z. Hong, X. Yang, S. Guo, Z. Wang, Z. Wang, C. Qian, X. Tang, H. Ji, and J. You, "MultiAgentBench: Evaluating the collaboration and competition of LLM agents," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.
- [29] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of Thoughts: Deliberate problem solving with large language models," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 11809–11822, 2023.
- [30] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, and T. Hoefler, "Graph of Thoughts: Solving elaborate problems with large language models," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, vol. 38, no. 16, pp. 17682–17690, 2024.
- [31] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and Y. Yao, "Reflexion: Language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 8634–8652, 2023.
- [32] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, S. Welleck, B. Bodnariu, S. Mishra, P. Sharma, K. Saraswat, and P. Clark, "Self-Refine: Iterative refinement with self-feedback," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 46534–46547, 2023.
- [33] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, M. Huang, and N. Duan, "CRITIC: Large language models can self-correct with tool interactive critiques," in *International Conference on Learning Representations (ICLR)*, 2024.

- [34] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multi-agent debate," *arXiv preprint arXiv:2305.14325*, 2023.
- [35] J. Zhang, J. X. Morris, and C. Yang, "Exchange-of-Thought: Enhancing large language model reasoning through peer-to-peer conversation," *arXiv preprint arXiv:2312.01823*, 2023.
- [36] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, A. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 68539–68551, 2023.
- [37] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, L. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *International Conference on Learning Representations (ICLR)*, 2024.
- [38] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," *arXiv preprint arXiv:2305.15334*, 2023.
- [39] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," in *International Conference on Learning Representations (ICLR)*, 2024.
- [40] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Narasimhan, and O. Press, "SWE-bench: Can language models resolve real-world GitHub issues?" in *International Conference on Learning Representations (ICLR)*, 2024.
- [41] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Bisk, D. Fried, U. Alon, and G. Neubig, "WebArena: A realistic web environment for autonomous agents," in *International Conference on Learning Representations (ICLR)*, 2024.
- [42] G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom, "GAIA: A benchmark for general AI assistants," in *International Conference on Learning Representations (ICLR)*, 2024.
- [43] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, J. Zhou, G. Li, and L. V. Gool, "OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments," *arXiv preprint arXiv:2404.07972*, 2024.
- [44] S. S. Y. Kim, J. Y. Koh, and D. Yang, "Evaluating operational risks and failure modes in multi-agent LLM systems," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3120–3135, 2024.
- [45] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, A. Zhu, X. Chen, X. Xie, C. Wang, L. Lu, and Y. Zhang, "A survey on evaluation of large language models," *ACM Computing Surveys*, vol. 56, no. 5, pp. 1–38, 2024.